



Deloitte



Java AWS developer

Bengaluru, Karnataka, India · 5 days ago · Over 100 people clicked apply

Promoted by hirer · Responses managed off LinkedIn

On-site

Full-time

Apply 

Save

Deloitte.

**Java Developer
2nd Technical
Round: **Toughest**
Scenario-Based
Interview
Questions for
4-6YOE**



www.prominentacademy.in



+91 98604 38743

Section 1 – Java Production Debugging

1.
After migrating your application from Java 11 to Java 21, API latency increased by 40% even though CPU utilization remained unchanged. How would you identify the root cause?
2.
A REST endpoint processes 30 requests/sec normally, but under 3,000 requests/sec it starts throwing intermittent `RejectedExecutionException`. How would you troubleshoot and redesign the solution?
3.
A production JVM generates frequent Full GCs every 8–10 minutes, but heap usage never exceeds 55%. What would you investigate?
4.
A multithreaded batch process occasionally skips records without throwing exceptions. How would you debug this issue?
5.
One API becomes slower after replacing imperative code with Java Streams. How would you determine whether Streams are the actual bottleneck?
6.
A thread dump shows hundreds of threads waiting on the same monitor. How would you identify the code responsible?
7.
Your application occasionally returns duplicate responses even though only one request was received. What possible concurrency issues would you investigate?
8.
How would you identify whether a performance issue originates from CPU contention, lock contention, or excessive context switching?
9.
A scheduled job overlaps with its previous execution during peak load. How would you redesign the scheduler?
10.
How would you safely process one million records in parallel without exhausting memory?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 1 – Java Production Debugging

11.

A developer replaces ConcurrentHashMap with HashMap for performance reasons. Production failures begin appearing only under load. How would you prove the root cause?

12.

How would you isolate memory retained by thread-local variables in a long-running application?

13.

Your service processes uploaded files concurrently, but one corrupted file causes other requests to slow down. How would you isolate failures?

14.

How would you identify hidden blocking operations inside asynchronous Java code?

15.

How would you benchmark two different Java implementations before deciding which one to deploy?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 2 – Spring Boot / Spring Cloud

16.

One Spring Boot service takes 20 seconds longer to start after enabling additional Spring Cloud components. How would you identify the startup bottleneck?

17.

A Config Server becomes unavailable during deployment. How should existing services behave?

18.

A service reads outdated configuration values even after configuration refresh. How would you troubleshoot?

19.

Your application exposes 150 APIs, but only three experience random latency spikes. How would you isolate the problem?

20.

A downstream service starts responding slowly. How would you prevent thread pool exhaustion in the calling service?

21.

How would you redesign synchronous service-to-service communication to reduce cascading latency?

22.

A single configuration mistake causes all application instances to fail during startup. How would you reduce deployment risk?

Pay After Placement

 Don't wait-call us at **+91 98604 38743** today

Section 2 – Spring Boot / Spring Cloud

23.

How would you implement request correlation across multiple Spring Boot services without modifying every controller?

24.

A circuit breaker opens too aggressively during temporary network glitches. How would you tune it?

25.

How would you safely introduce a breaking API change across dozens of dependent microservices?

26.

A service occasionally receives duplicate REST requests due to client retries. How would you ensure business operations execute only once?

27.

How would you identify unnecessary object creation inside Spring Boot request processing?

28.

How would you migrate a large Spring MVC application into smaller Spring Cloud services incrementally?

29.

One microservice consumes significantly more memory than others with similar traffic. What areas would you investigate?

30.

How would you verify that Spring dependency injection is not causing unintended shared state?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 3 – AWS Architecture & Cloud Migration

31.

Your application is being migrated from an on-premise data center to AWS. Which components would you migrate first and why?

32.

An EC2 instance becomes unreachable even though CloudWatch shows healthy CPU and memory. How would you investigate?

33.

Your application stores uploaded documents in S3. A deployment accidentally changes bucket permissions and users receive Access Denied errors. How would you recover?

34.

An RDS failover occurs during peak traffic. What should your application do to recover automatically?

35.

How would you migrate terabytes of historical data into AWS without affecting production users?

36.

A Lambda function processing invoices exceeds its execution timeout. How would you redesign the workflow?

37.

API Gateway latency suddenly doubles after introducing request validation. How would you determine whether API Gateway is the bottleneck?

38.

How would you securely allow one AWS account to access S3 objects stored in another AWS account?

39.

An application running on EC2 needs temporary AWS credentials. Why would you prefer IAM Roles over access keys?

40.

How would you detect whether AWS network latency or application latency is causing slow responses?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 3 – AWS Architecture & Cloud Migration

41.

A cloud migration increases monthly infrastructure cost by 60%. How would you identify optimization opportunities?

42.

An Auto Scaling Group launches new EC2 instances, but traffic never reaches them. What would you investigate?

43.

How would you design a highly available file-upload service using EC2, S3, and RDS?

44.

How would you migrate a monolithic application to AWS while minimizing downtime?

45.

A Lambda function updates RDS directly and causes connection exhaustion. How would you redesign it?

46.

How would you handle S3 eventual consistency concerns for newly uploaded files that must be immediately accessible?

47.

A deployment accidentally deletes an important S3 object. Which AWS features would help recover it?

48.

Your RDS instance reaches maximum connections during traffic spikes. How would you troubleshoot?

49.

How would you validate cloud migration success beyond simply checking application availability?

50.

A production workload suddenly starts using significantly more EC2 instances than expected. How would you determine whether scaling is legitimate or caused by an application issue?

Pay After Placement

 Don't wait-call us at **+91 98604 38743** today

Section 4 – Docker / Kubernetes / EKS

51.

A Docker container works locally but repeatedly crashes after deployment to Kubernetes. How would you isolate the issue?

52.

A Kubernetes Deployment reports all Pods as Ready, but customers receive HTTP 503 errors. What possible causes would you investigate?

53.

Your application stores uploaded files inside the container filesystem. What production problems could occur?

54.

A container restarts every few minutes because of failed liveness probes, even though the application eventually starts correctly. How would you solve this?

55.

How would you deploy a new version without terminating long-running customer requests?

56.

A Pod repeatedly enters CrashLoopBackOff only in production. What logs and events would you inspect first?

57.

How would you safely roll back an unsuccessful Kubernetes deployment while preserving user sessions?

58.

A memory leak appears only after containers have been running for several days. How would you investigate?

59.

One Pod consistently receives more traffic than others behind the same Service. How would you troubleshoot?

60.

How would you safely rotate application secrets in Kubernetes without downtime?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 4 – Docker / Kubernetes / EKS

61.

A deployment succeeds, but newly created Pods continue using an old Docker image. What could cause this?

62.

How would you migrate an application from standalone EC2 deployments to EKS incrementally?

63.

A node failure terminates several Pods simultaneously. How would you ensure minimal customer impact?

64.

How would you identify resource limits causing CPU throttling inside Kubernetes?

65.

How would you investigate intermittent networking failures between Pods running on different nodes?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 5 – SQL & Distributed Data

66.

A PostgreSQL query becomes slow only after the table grows beyond 200 million rows. How would you optimize it?

67.

How would you redesign a database schema to reduce locking during heavy write operations?

68.

A long-running reporting query impacts OLTP transaction performance. How would you separate workloads?

69.

How would you migrate database schema changes with zero downtime?

70.

A transaction deadlock occurs only during Black Friday traffic. How would you reproduce and analyze it?

71.

How would you identify whether connection pooling is causing database latency?

72.

A database index improves reads but significantly slows writes. How would you evaluate the trade-off?

73.

How would you safely archive historical data while keeping reporting accurate?

74.

A read replica serves stale data during a critical business process. How would you decide when to read from the primary instead?

75.

How would you validate database performance improvements before deploying to production?

Pay After Placement

 Don't wait-call us at **+91 98604 38743** today

Section 6 – Microservices Performance & Reliability

76.

A single slow downstream dependency increases latency across the entire platform. How would you isolate the impact?

77.

How would you determine whether synchronous or asynchronous communication is better for a specific business workflow?

78.

One microservice generates excessive retries that overload another service. How would you redesign retry behavior?

79.

How would you identify hidden network latency between microservices deployed in different AWS Availability Zones?

80.

A service remains healthy individually but causes overall system slowdown when combined with others. How would you investigate?

81.

How would you prevent one noisy tenant from affecting all customers in a shared microservice?

82.

A service returns responses within SLA, but customers still perceive the application as slow. What additional factors would you investigate?

83.

How would you measure the impact of introducing response caching into a distributed system?

84.

How would you safely introduce compression for REST APIs without degrading CPU performance?

85.

A downstream dependency introduces intermittent 2-second delays. How would you prevent request queues from growing uncontrollably?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Section 6 – Microservices Performance & Reliability

86.

How would you identify whether serialization/deserialization is affecting API performance?

87.

How would you verify that a microservice can survive the sudden loss of one AWS Availability Zone?

88.

How would you test resilience against intermittent packet loss between services?

89.

A production deployment passes all functional tests but immediately fails under real traffic. What types of testing were likely missing?

90.

How would you prove that an optimization actually improved end-to-end customer response time rather than just reducing server processing time?

Pay After Placement

 Don't wait-call us at **+91 98604 38743** today

Final Technical Leadership Scenarios

91.

You are asked to review the architecture of a cloud-native Java application before production go-live. What technical risks would you evaluate first?

92.

A client requests migration to AWS within three months, but the existing application contains tightly coupled modules and hardcoded infrastructure dependencies. How would you plan the migration?

93.

A critical production incident spans Java code, Kubernetes, and AWS networking simultaneously. How would you coordinate investigation across teams?

94.

How would you prioritize fixing technical debt versus delivering new business features in a fast-paced Agile project?

95.

You inherit a microservice ecosystem with inconsistent API standards across teams. How would you standardize them without disrupting delivery?

96.

A performance optimization improves throughput but increases cloud costs significantly. How would you evaluate whether the change should be accepted?

97.

How would you design a deployment strategy that allows immediate rollback while preserving in-flight customer requests?

98.

A customer reports intermittent failures that cannot be reproduced in any non-production environment. How would you build a structured investigation plan?

99.

If you had to identify the single highest architectural risk before releasing a large Java microservices platform, what evidence would you collect to support your assessment?

100.

Imagine you are the technical owner of this Deloitte project. What architecture, operational, scalability, observability, and cloud-readiness improvements would you prioritize during the next six months, and why?

Pay After Placement

 **Don't wait-call us at +91 98604 38743 today**

Prominent Academy me:

- ✓ Real production scenarios
- ✓ Pressure-based **mock interviews**
- ✓ Decision-making framework
- ✓ Interview Prep
- ✓ Interview calls (Unlimited Till placement)
- ✓ **Pay After Placement option**



WhatsApp



+91 98604 38743